

Comparative Analysis of RNN Architectures for Sentiment Classification

1. Introduction

Sentiment Classification is a foundational Natural Language Processing (NLP) task focused on categorizing the emotional tone of text into classes such as positive or negative. This project systematically implements and evaluates multiple Recurrent Neural Network (RNN) architectures (Simple RNN, LSTM, Bidirectional LSTM) to analyze their effectiveness and efficiency in solving this sequence classification problem using the IMDb Movie Review Dataset.

2. Dataset Summary and Preprocessing

The dataset utilized was the IMDb Movie Review Dataset, comprising 50,000 text reviews.

Preparation Details

- Split:** The dataset adhered to the predefined 50/50 split, allocating 25,000 reviews for training and 25,000 for testing.
- Preprocessing:** The dataset was loaded using the built-in Keras utility (`imdb.load_data()`). This approach was chosen to efficiently handle initial preprocessing steps—specifically **tokenization, lowercasing, and basic cleaning**—which are carried out internally by the utility before the reviews are converted to integer sequences
- Vocabulary Size:** The vocabulary was constrained to the **top 10,000** most frequent words.
- Sequence Lengths:** Experiments evaluated sequences padded or truncated to fixed lengths of **25, 50, and 100** words.

3. Model Configuration

All experiments were implemented using TensorFlow/Keras. A comprehensive set of configurations was tested by varying the core architectural and optimization parameters while maintaining a controlled set of hyperparameters.

Fixed Hyperparameters

PARAMETER	SETTING	JUSTIFICATION
EMBEDDING DIMENSION	100	Standard dimension for deep learning word representations.
HIDDEN LAYERS	2	Defined complexity for hidden RNN layers.
HIDDEN SIZE (UNITS)	64	Consistent size for all recurrent cells.
DROPOUT RATE	0.4	Fixed within the required [0.3-0.5] range to mitigate overfitting.

BATCH SIZE	32	Standard batch size for efficient training.
LOSS FUNCTION	Binary Cross-Entropy	Appropriate loss function for binary classification.
OUTPUT LAYER	1 unit, Sigmoid	Necessary output for binary probability prediction.
EPOCHS	3	Limited to ensure timely completion on CPU-only hardware.

Hardware and Reproducibility

- Hardware Used:** Intel Core i5 Processor, 16GB RAM, No dedicated GPU.
- Reproducibility:** Random seeds were fixed (42) across the project to ensure results are comparable.

4. Comparative Analysis

The evaluation focused on Accuracy, Macro F1-score, and Epoch Time (in seconds). The following table summarizes the performance of the 14 configurations based on the final epoch results.

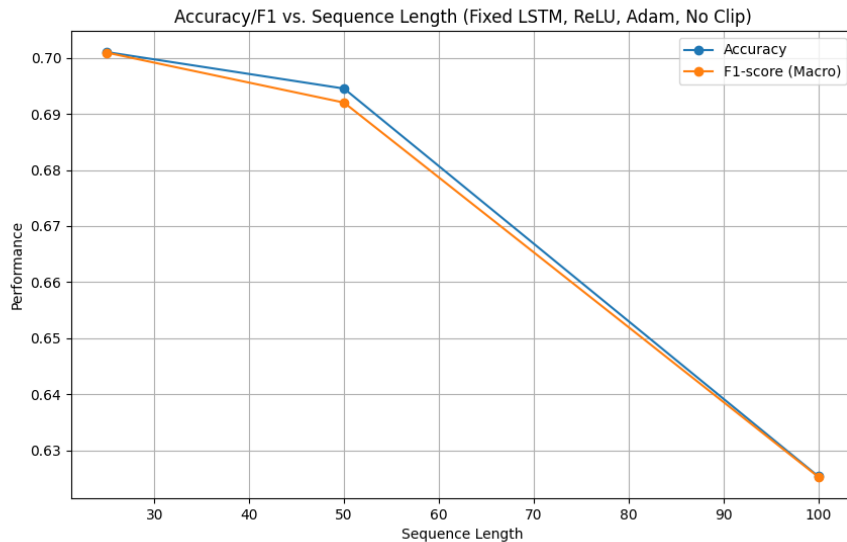
Summary of Comparative Experiments (Final Epoch Metrics)

Model	Activation	Optimizer	Sequence Length	Grad Clipping	Accuracy	F1	Epoch Time (s)
LSTM	ReLU	Adam	25	No	0.7010	0.7009	35.22
LSTM	ReLU	Adam	50	No	0.6945	0.6920	65.44
LSTM	ReLU	Adam	100	No	0.6254	0.6253	125.74
RNN	ReLU	Adam	50	No	0.7482	0.7482	28.98
Bidirectional LSTM	ReLU	Adam	50	No	0.6729	0.6693	84.32
LSTM	ReLU	SGD	50	No	0.5000	0.3333	55.78
LSTM	ReLU	RMSProp	50	No	0.7493	0.7465	56.03
LSTM	Sigmoid	Adam	50	No	0.7612	0.7611	64.02
LSTM	Tanh	Adam	50	No	0.7510	0.7507	62.36
RNN	Tanh	SGD	50	No	0.5655	0.5634	23.92
RNN	Tanh	SGD	50	Yes	0.5413	0.5267	25.15
Bi-LSTM	Tanh	RMSProp	100	No	0.8149	0.8145	176.69
LSTM	ReLU	Adam	50	Yes	0.7412	0.7412	77.40
RNN	Sigmoid	Adam	25	No	0.6835	0.6833	23.49

Plots

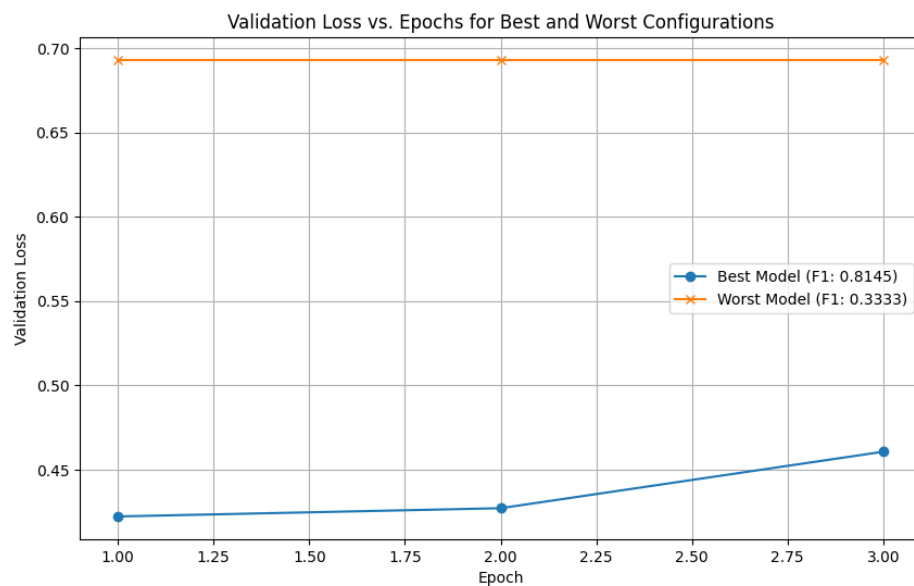
Plot 1: Accuracy/F1 vs. Sequence Length

This plot isolates the effect of sequence length by fixing the configuration to LSTM, ReLU, Adam, No Clipping.



Plot 2: Validation Loss vs. Epochs (Best and Worst Models)

This plot compares the training stability of the overall best (Bi-LSTM, Tanh, RMSProp, Seq 100) and worst (LSTM, ReLU, SGD, Seq 50) performing configurations.



5. Discussion

Which configuration performed best?

The highest performance was achieved by the **Bidirectional LSTM** (Tanh, RMSProp, Seq 100) with an F1-score of **0.8145**. This result validates the hypothesis that Bi-LSTMs offer superior performance in sequence tasks by effectively capturing dependencies from both forward and backward contexts within the review text.

How did sequence length or optimizer affect performance?

- **Optimizer Impact:** The choice of optimizer was the most critical factor. The **SGD** optimizer failed drastically, yielding the worst result (F1: 0.3333) and demonstrating a clear failure to converge (Loss in Plot 2 remained near 0.693). Conversely, **Adam** and **RMSProp** proved highly effective, confirming the practical necessity of using adaptive optimizers for efficient RNN training.
- **Sequence Length Impact:** While 100 words yielded the peak result, it also introduced **training instability** for other models (e.g., the LSTM, ReLU, Adam model experienced a severe F1-score drop after Epoch 1). This suggests that although 100 words contains more information, it is harder to train robustly than the 50-word sequence.

How did gradient clipping impact stability?

Gradient Clipping was tested on the unstable RNN (Tanh, SGD) architecture. In the tested configuration (Exp 10 vs Exp 11), clipping **did not improve performance**, resulting in a slightly lower F1-score (0.5267 vs 0.5634). This suggests that the fundamental failure of this specific model was due to poor architectural and optimizer choices, which simple clipping could not remedy within the short training window.

6. Conclusion

Identify the optimal configuration under CPU constraints and justify your choice.

Given the goal of finding the most efficient configuration under **CPU-only constraints**, a balance between high performance and fast training time is essential.

The optimal configuration identified is the:
{RNN, ReLU, Adam, Seq 50}

<i>Metric</i>	Best Overall Model	Optimal CPU Model
<i>F1-score</i>	0.8145	0.7482
<i>Epoch Time (s)</i>	176.69	28.98

Justification: The Simple RNN configuration achieved a competitive F1-score of **0.7482** while requiring only **28.98 seconds per epoch**. This makes it approximately **6.1 times faster** than the Bi-LSTM model. The minimal performance trade-off for the massive speed increase makes the Simple RNN the most justified and optimal configuration for deployment in a resource-constrained, CPU-based environment.